

머신러닝 프로젝트 (RISE) · 3주차

웹/앱 프로토타입 구현

"화면 세 개, 기능 하나, 시연 가능한 앱 하나 — 오늘 안에"

- 화면 설계 → 레이아웃 상태 → 기능 연결 → 실행·시연 준비 → 내 주제 확정
- 오늘의 앱: 이탈 위험 체크 서비스 (3단계에서 ML API를 붙일 프론트엔드)
- 오늘의 결과물: 다중 화면 앱 · 내 앱 주소에서 3화면 동작 · 프로젝트 기획서 1장

학습목표와 오늘의 경로

이론 35분 + 실습 70분 + 주제 워크숍 15분 — 실습은 한 앱을 처음부터 끝까지

학습목표

- 프로토타입의 범위를 **Must 기능 3개·화면 3~5개**로 정하고 화면 설계를 프롬프트로 옮긴다
- Streamlit의 **재실행 모델·session_state·캐시**를 이해하고 레이아웃을 구성한다
- 입력 → 처리 함수 → 출력의 **데이터 흐름**을 만들고 로직을 화면에서 분리한다
- 앱 전용 터미널에서 실행해 **내 앱 주소**에서 시연 가능한 상태를 만들고, 로컬 Git으로 되는 버전을 남긴다

시간	내용	형태
0:00	과제 리뷰 (앱 3개 화면에) · 오늘의 목표	10분
0:10	PART 1 프로토타입·화면 설계 · LAB 3-1	강의 10 + 실습 5
0:25	PART 2 화면 구성 · LAB 3-2 레이아웃 · 상태·캐시	강의 10 + 실습 15
0:50	PART 3 기능 연결 · LAB 3-3 폼→점수 · 3-4 데이터 화면 · 3-5 다중 페이지	강의 10 + 실습 30
1:30	PART 4 실행·시연 준비 · 로컬 Git · LAB 3-6 앱 전용 터미널	강의 5 + 실습 15
1:50	PART 5 내 주제 워크숍(짝 점검) · 정리 · 과제	15분

실습 환경

준비물: 서버 환경(내 계정 · ~/work) 그대로. 실습 파일은 `cp -r ~/work/shared/week3/* ~/work/(layout_demo.py, churn_check/)`. Git은 서버에 있음 — 계정 불필요. 주제 워크숍용 `templates/01_프로젝트_기획서_1page.md`.

과제 리뷰 — 지난주 앱 세 개를 화면에

10분. 보는 관점은 "동작하나 · 프롬프트 로그에 검증이 적혀 있나 · 틀렸던 사례가 있나"

자주 나온 문제	원인	오늘 어디서 해결
슬라이더를 움직이면 입력이 사라진다	위젯을 만질 때마다 스크립트 전체 재실행 → 변수 초기화	PART 2 <code>session_state</code>
파일을 올릴 때마다 느리다	재실행마다 CSV를 다시 읽음	PART 2 <code>@st.cache_data</code>
한 파일이 200줄이 넘어 AI가 엉뚱한 곳을 고친다	화면과 로직이 섞임, 한 파일에 여러 화면	PART 3 <code>core.py</code> 분리 · <code>pages/</code>
다른 폴더에서 실행하면 안 돈다	실행 위치·상대 경로	PART 4 실행 위치 · 상대 경로

오늘의 앱 "**이탈 위험 체크 서비스**": 홈 / 데이터 탐색 / 위험 체크 3화면. 위험 점수는 지금은 규칙(가짜 백엔드)이고, 2단계에서 학습한 모델로, 3단계에서 REST API 호출로 바뀝니다. 오늘 만든 화면은 그때 그대로 씁니다.

01

PART 1

프로토타입과 화면 설계

완성품이 아니라 "질문에 답하는 최소한의 물건" — 무엇을 빼느냐가 설계다

이 파트에서

- 1 프로토타입의 목적과 범위 (MVP·Must 3개)
- 2 앱의 3층: 입력 · 처리 · 출력
- 3 화면 목록과 텍스트 와이어프레임
- 4 LAB 3-1 오늘 앱의 화면 설계 → 프롬프트

프로토타입이란 — 답하려는 질문이 먼저다

"이 서비스가 쓸모 있나?"에 5분 시연으로 답할 수 있는 최소한. 5주차 시연 = 프로토타입의 정의

넣는 것 (Must)

- 핵심 사용자 스토리 **3개 이하**
- 실제 데이터 또는 실제 같은 샘플
- 입력 → 결과가 **눈에 보이는** 흐름 1개
- 시연 가능 상태 (내 앱 주소에서 3화면 동작 — 5주차는 자기 자리에서 시연)

미루는 것 (Should / Could)

- 로그인·회원가입
- 예쁜 디자인·애니메이션
- 모든 예외 처리 (Must 흐름의 예외만)
- 진짜 ML 모델 (2단계에서)

안 하는 것 (Won't)

- 데이터베이스·결제·알림
- 모바일 네이티브 앱
- 다국어·접근성 전면 대응
- "나중에 필요할지도" 기능 전부

실무에서 이렇게 틀린다 ①

프로토타입이 실패하는 첫째 이유는 기술이 아니라 **범위**다. 4주차 워크숍에서 "Must가 3개를 넘는 기획서"는 교수 피드백에서 반드시 줄이게 된다. 지금부터 3개로.

앱의 3층 — 입력 · 처리 · 출력, 그리고 흐름

어떤 앱이든 이 셋으로 그려진다. 처리 층을 함수로 분리하면 테스트·API 교체가 쉬워진다

1

입력 (위젯)

selectbox · number_input

2

검증

빈 값 · 범위 · 타입

3

처리 (함수)

core.churn_score(...)

4

출력 (위젯)

metric · chart · dataframe

5

상태

session_state · cache

화면

```
# 화면 파일 (pages/2_이탈_위험_체크.py) — 입력·출력만
with st.form("check"):
    contract = st.selectbox("계약 유형", [...])
    ...
    submitted = st.form_submit_button("위험도 계산")
if submitted:
    r = churn_score(contract, tenure, monthly, internet, support)
    st.metric("위험 점수", r["score"])
```

로직

```
# 로직 파일 (core.py) — Streamlit을 모른다
def churn_score(contract, tenure, monthly, internet, support) -> dict:
    score, reasons = 10, []
    if contract == "Month-to-month":
        score += 35; reasons.append("월 단위 계약 (+35)")
    ...
    return {"score": score, "level": level, "reasons": reasons}

# 3단계: 이 함수 호출 → requests.post(API_URL, json=...)
```

둘 곳	무엇	왜
화면 (pages/)	위젯 · 레이아웃 · 메시지 · session_state	사용자 경험은 자주 바뀐다 — 바뀌도 로직이 안 깨지게
로직 (core.py)	점수 계산 · 검증 규칙 · 데이터 변환 함수	pytest로 검증, 3단계에서 FastAPI로 그대로 이식
데이터 (data/)	CSV · 샘플 · (나중에) 모델 파일	경로 한 곳에서 관리, 앱 폴더 안에 함께

규칙: **core.py**는 **import streamlit**을 하지 않는다. 그래야 pytest로 테스트되고(2주차), FastAPI에 그대로 옮길 수 있다(12주차). AI에게 프롬프트를 쓸 때도 "로직은 core.py, 화면은 pages/"라고 제약을 명시.

화면 목록과 텍스트 와이어프레임

그림 도구 없이도 된다 — 화면마다 "영역 · 위젯 · 데이터"를 글로 적으면 그대로 프롬프트가 된다

설계 텍스트 와이어프레임

[앱] 이탈 위험 체크 서비스 · 대상: 통신사 고객관리팀

[화면 목록]

- 0 홈 서비스 소개, 화면 안내
- 1 데이터 탐색 필터(계약·가입개월) → 지표 3개 → 차트 2개
- 2 위험 체크 폼(5입력) → 점수·수준·근거 → 조회 이력

[화면 2 와이어프레임 - 위→아래 순서]

제목 이탈 위험 체크

[폼] 계약유형▼ 가입개월# | 월요금# 인터넷▼
기술지원 (예/아니오) [위험도 계산]

결과 점수 NN/100 · 수준(색) · 진행바 · 근거 ...

이력 표 (세션 누적) [이력 지우기]

영역

적을 것

화면 목록	이름 · 한 줄 목적 · 들어가는 스토리(US-ID)
영역 배치	위→아래, 왼→오른쪽 순으로. 사이드바는 필터·설정
위젯	종류(▼ 선택, # 숫자, □ 체크, [버튼])와 범위·기본값
데이터	어디서 오나(파일·입력·API), 어떤 열을 쓰나
상태	재실행 후에도 남아야 하는 것 (이력·선택값)

왜 글로 쓰나

와이어프레임을 프롬프트 [맥락]에 그대로 넣으면 AI가 위젯 종류·순서를 추측하지 않는다. 손으로 그린 사진을 넣을 수 있는 도구(ChatGPT·Claude·Gemini)도 같은 효과.

LAB 3-1 · 오늘 앱의 화면 설계를 프롬프트로

위 와이어프레임 형식으로 화면 1(데이터 탐색)을 설계하고, "계획" 패턴으로 파일 구조를 받는다 (5분)

계획 프롬프트

너는 Streamlit 개발자다. 코드는 아직 쓰지 마라.
 [목표] 아래 화면 목록.와이어프레임대로 3화면 앱
 [맥락] Python 3.11, streamlit≥1.46, pandas.
 데이터 data/telco_churn.csv (7,043행, 열: Contract, tenure, MonthlyCharges, Churn(Yes/No) ...)
 로직은 core.py, 화면은 app.py + pages/
 [요청] ① 파일 구조 ② 구현 순서(작은 단위 5단계 이하)
 ③ 각 단계의 확인 방법
 [출력] 표 하나

#	단계 (AI 응답 예)	확인
1	app.py 홈 + pages/ 빈 파일 2개	메뉴에 3화면 뜸
2	core.churn_score + 단위 테스트	pytest 통과
3	화면 2: 폼 → 점수 → 이력	예시 입력 90점
4	화면 1: 로드-캐시-필터-지표	고객 수 7,043
5	화면 1: 차트 2개 · 원본 보기	계약별 이탈률 막대

해 보기: 내 주제가 정해졌다면 그 앱으로. 없으면 2주차 LAB 2-2 아이디어로. **화면 1개의 와이어프레임 + 계획 표**를 받으면 LAB 끝 — 오늘 남은 LAB은 이 계획의 3→4→5단계를 실제로 밟는다.

02

PART 2

화면 구성

레이아웃 부품 다섯 개, 그리고 Streamlit이 값을 잃어버리는 이유와 해법

이 파트에서

- 1 위젯 카탈로그 — 입력·출력·레이아웃
- 2 LAB 3-2 레이아웃 데모
(`sidebar.columns.tabs.metric.expander`)
- 3 재실행 모델과 `session_state`
- 4 캐시 — `@st.cache_data`

위젯 카탈로그 — 이것만 알면 프로토타입은 된다

전체 목록은 docs.streamlit.io "API reference". 프롬프트에 위젯 이름을 명시하면 결과가 안정된다

층	위젯	한 줄 용법
입력	text_input number_input slider selectbox multiselect radio checkbox date_input file_uploader form	값을 변수로 받는다. form은 제출 버튼을 누를 때만 한 번 실행
출력	write markdown metric dataframe table bar_chart line_chart scatter_chart progress success/warning/error/info	write는 무엇이든 그린다. 표는 dataframe(스크롤·정렬), 지표는 metric
레이아웃	sidebar columns tabs expander container set_page_config	사이드바 = 필터·설정, 컬럼 = 지표 나란히, 탭 = 화면 안 화면
상태·성능	session_state cache_data cache_resource rerun stop	재실행에서 살아남기 · 다시 계산 안 하기
다중 화면	pages/ 폴더 (파일명 순서 = 메뉴 순서)	pages/1_데이터_탐색.py → 메뉴 "데이터 탐색"

실무에서 이렇게 틀린다 ②

AI가 자주 만들어 내는 없는 위젯: `st.chart_bar`, `st.input_text`, `st.cache`(구버전). 실행하면 `AttributeError`로 바로 드러난다 — 오류 전체를 되돌려 주면 한 바퀴에 고쳐진다.

LAB 3-2 · 레이아웃 데모 — 부품 5개를 한 화면에

sidebar · columns · tabs · metric · expander. 실행해 보고 각 부품이 어디에 그려지는지 확인 (10분)

LAB 3-2 practice/week3/layout_demo.py

```
st.set_page_config(page_title="레이아웃 데모", layout="wide")

with st.sidebar:                                # ① 사이드바 = 설정·필터
    st.header("설정")
    n = st.slider("표시 행 수", 5, 50, 10)
    show_raw = st.checkbox("원본 표 보기", value=True)

st.title("레이아웃 데모")
c1, c2, c3 = st.columns(3)                      # ② 상단 지표 3칸
c1.metric("고객 수", "7,043")
c2.metric("이탈률", "26.5%", "-1.2%p")
c3.metric("평균 월요금", "$64.8")

tab1, tab2 = st.tabs(["표", "설명"])           # ③ 탭
df = pd.read_csv("../data/telco_churn.csv")
with tab1:
    if show_raw:
        st.dataframe(df.head(n), width="stretch")
with tab2:
    with st.expander("열 목록 펼치기"):         # ④ 접히는 영역
        st.write(list(df.columns))
```

해 보기

프롬프트 힌트

컬럼 비율 2:1:1

"columns([2,1,1])로 첫 지표를 크게"

탭 하나 추가 — 차트

"세 번째 탭에 tenure 히스토그램 st.bar_chart"

사이드바 필터 → 표에 반영

"사이드바 multiselect로 Contract를 고르면 표가 필터되게"

metric delta 색 반전

"delta_color='inverse'로 이탈률 감소를 초록으로"

함정 하나

layout="wide"는 첫 번째 **Streamlit 명령**이어야 한다. AI가 중간에 넣으면 **StreamlitAPIException** — 이것도 오류 전체를 붙여넣으면 스스로 고친다.

```
streamlit run layout_demo.py --server.address 0.0.0.0 --
server.port 8000
```

<https://ahnbi4.suwon.ac.kr:5300/user/YOURID/proxy/8000/>

재실행 모델과 session_state — "값이 사라져요"의 해법

Streamlit은 위젯을 만질 때마다 스크립트를 처음부터 다시 실행한다. 변수는 매번 새로 태어난다

문제

```
# 안 됨 - 버튼을 누를 때마다 history가 []로 초기화
history = []
if st.button("추가"):
    history.append(value)      # 늘 1개
```

해법

```
# 됨 - session_state는 브라우저 탭이 살아 있는 동안 유지
if "history" not in st.session_state:
    st.session_state.history = []

if st.button("추가"):
    st.session_state.history.append(value)

st.dataframe(pd.DataFrame(st.session_state.history))
if st.button("지우기"):
    st.session_state.history = []
    st.rerun()                # 즉시 다시 그리기
```

상황

도구

재실행 사이에 값 유지 (이력·선택)

`st.session_state["key"]`

여러 입력을 모아 한 번에 처리

`st.form + form_submit_button`

버튼을 누른 "그 순간"만 참

`st.button` — 다음 재실행에서는 False

위젯 값을 코드로 바꾸기

`key=`를 주고 `session_state[key] = ...`

탭을 닫으면

세션 종료 → 상태 소멸 (영구 저장은 파일·DB)

실무에서 이렇게 툰다 ③

버튼 안에 결과를 그리면 다음 클릭에서 사라진다. 결과를 `session_state`에 넣고 버튼 밖에서 그린다 — LAB 3-3 이력 표가 그 구조.

캐시 — 같은 파일을 매번 다시 읽지 않기

재실행 모델의 비용은 "느림". 무거운 함수에 @st.cache_data 한 줄

캐시

```
@st.cache_data          # 같은 인자 → 저장된 결과 반환
def load(path: str) -> pd.DataFrame:
    df = pd.read_csv(path)
    df["TotalCharges"] = pd.to_numeric(df["TotalCharges"],
                                      errors="coerce")

    return df

df = load("data/telco_churn.csv") # 두 번째부터 즉시
```

cache_data 결과가 **데이터**(DataFrame·리스트·dict)인 함수. 복사본을 돌려주므로 안전

cache_resource 결과가 **연결·모델**(DB 연결, 학습된 모델 객체)인 함수. 3단계에서 모델 로드 사용

무효화 인자가 바뀌면 다시 계산. 파일 내용만 바뀌면 모름 → `ttl=600`(초) 또는 앱 메뉴 "Clear cache"

업로드 파일 `file_uploader` 객체는 매번 다르므로 캐시 대상 아님 — 경로 기반 로드만 캐시

프롬프트에 넣을 한 줄: "데이터 로드 함수는 @st.cache_data로, 파일 경로는 앱 실행 위치 기준 상대 경로(data/...)로". 경로를 절대 경로(/data/project/.../5207/...)로 쓰면 폴더를 옮기는 순간 깨진다.

03

PART 3

기능 연결

입력 폼에서 결과까지 — 그리고 화면을 여러 개로 나누기

이 파트에서

- 1 LAB 3-3 폼 → 처리 함수 → 결과 · 이력
- 2 LAB 3-4 데이터 탐색 화면 (로드·필터·지표·차트)
- 3 LAB 3-5 다중 페이지 구조 pages/
- 4 외부 API 호출 맛보기 — 3단계 예고

LAB 3-3 · 폼 → 처리 함수 → 결과 · 이력

2주차에 쓴 프롬프트(명세 US-1)를 실제로 실행한다. core.py는 주어진 것을 쓴다 (12분)

LAB 3-3 churn_check/pages/2_이탈_위험_체크.py (앞부분)

```
from core import churn_score

if "history" not in st.session_state:
    st.session_state.history = []

with st.form("check"):
    c1, c2 = st.columns(2)
    contract = c1.selectbox("계약 유형", ["Month-to-month", "One year", "Two year"])
    tenure = c1.number_input("가입 개월", 0, 72, 3)
    monthly = c2.number_input("월요금 ($)", 18.0, 120.0, 85.0, step=0.5)
    internet = c2.selectbox("인터넷", ["DSL", "Fiber optic", "No"])
    support = st.radio("기술지원 가입", ["Yes", "No"], horizontal=True)
    submitted = st.form_submit_button("위험도 계산")

if submitted:
    r = churn_score(contract, int(tenure), float(monthly), internet, support)
    color = {"높음": "red", "보통": "orange", "낮음": "green"}[r["level"]]
    st.markdown(f"### 위험 점수  $\frac{r['score']}{100} \times 100 = {r['score']}$  : {color}[r['level']]")
    st.progress(r["score"] / 100)
    for why in r["reasons"]:
        st.write("•", why)
    st.session_state.history.append({"계약": contract, "개월": tenure,
                                     "월요금": monthly, "점수": r["score"], "수준": r["level"]})
```

확인

기대

기본값으로 제출

85점 "높음", 근거 4줄

Two year · 60개월 · 30달러 ·
DSL · Yes

10점 "낮음", 근거 없음

세 번 제출

이력 표 3행 · "이력 지우기" 후 0행

폼 안에서 값만 바꿈

제출 전에는 **아무 일도 없음** (form의 성질)

해 보기 (프롬프트로): ① 점수 70 이상이면 `st.error`로 "리텐션 쿠폰 대상" 표시 ② 이력 표에 조회 시각 열 추가 (`datetime.now()`) ③ 이력을 CSV로 내려받는 `download_button`. 한 바퀴에 하나씩, 되는 버전을 남기고.

LAB 3-4 · 데이터 탐색 화면 — 로드 · 필터 · 지표 · 차트

사이드바 필터가 바뀌면 지표와 차트가 따라 바뀐다. 캐시된 로드 함수로 시작 (12분)

LAB 3-4 churn_check/pages/1_데이터_탐색.py (핵심부)

```
up = st.sidebar.file_uploader("다른 CSV 올리기 (선택)", type="csv")
df = pd.read_csv(up) if up is not None else load("data/telco_churn.csv")

with st.sidebar:
    contracts = st.multiselect("계약 유형", sorted(df["Contract"].unique()),
                               default=list(df["Contract"].unique()))
    t_min, t_max = st.slider("가입 개월", 0, int(df["tenure"].max()),
                             (0, int(df["tenure"].max())))

view = df[df["Contract"].isin(contracts) & df["tenure"].between(t_min, t_max)]
churn_rate = (view["Churn"] == "Yes").mean() if len(view) else 0.0

c1, c2, c3 = st.columns(3)
c1.metric("고객 수", f"{len(view):,}")
c2.metric("이탈률", f"{churn_rate:.1%}")
c3.metric("평균 월요금", f"${view['MonthlyCharges'].mean():.1f}" if len(view) else "-")

st.subheader("계약 유형별 이탈률")
rate = view.groupby("Contract")["Churn"].apply(lambda s: (s == "Yes").mean())
st.bar_chart(rate.rename("이탈률"))
```

확인

기대

필터 없이	7,043명 · 26.5% · \$64.8
Month-to-month만	3,875명 · 42.7%
가입 0~6개월	이탈률 50% 안팎 (초기 이탈)
필터로 0명	지표 0 · "-" — 오류 없이

실무에서 이렇게 틀린다 ④

if len(view) else ... 같은 빈 결과 처리를 AI는 자주 빼먹는다. 프롬프트 [제약]에 "필터 결과
과가 비어도 오류 없이"를 넣거나, 비워 보고 오류를 되돌려 준다. 5주차 루브릭 "예외 입력도 처리"가 이 부분.

LAB 3-5 · 다중 페이지 — pages/ 폴더 하나로 메뉴가 생긴다

홈 + 화면 2개를 폴더 구조로. 실행은 `streamlit run app.py` 한 번 (6분)

구조 practice/week3/churn_check/

```
churn_check/
├─ app.py                # 홈 (진입점)
├─ core.py               # 로직 - streamlit 없음
├─ pages/
│   └─ 1_데이터_탐색.py  # 메뉴 순서 = 파일명 앞 숫자
│       └─ 2_이탈_위험_체크.py
├─ data/telco_churn.csv  # 상대 경로 기준 = app.py 위치
├─ requirements.txt      # streamlit>=1.40, pandas>=2.0 (기록용)
├─ .gitignore            # __pycache__/ secrets.toml app.log
└─ README.md             # 실행법 · 구조 · 앱 주소

$ cd churn_check
$ streamlit run app.py    # 왼쪽 메뉴: app / 데이터 탐색 / 이탈 위험 체크
```

파일명 규칙 숫자_이름.py — 숫자는 순서, 밑줄은 공백으로 메뉴에 표시

공유 코드 `core.py`처럼 루트에 두면 모든 페이지에서 `from core import ...`

공유 상태 `session_state`는 페이지를 오가도 유지 — 화면 2의 이력을 화면 1에서도 볼 수 있다

실행 위치 항상 `app.py`가 있는 폴더에서 실행. 데이터 경로는 여기 기준

해 보기: `pages/3_소개.py`를 프롬프트로 추가 — "문제·사용자·데이터 출처·AI 활용 기록을 보여 주는 페이지". 4주차 내 앱에도 이 페이지가 들어간다.

외부 API 호출 맛보기 — 3단계의 모양을 미리 본다

지금 `churn_score()` 한 줄이 12주차에 `requests.post()` 한 줄로 바뀐다. 화면 코드는 그대로

지금

```
# 지금 (3주차): 같은 프로세스 안의 함수
from core import churn_score
r = churn_score(contract, tenure, monthly, internet, support)
```

나중

```
# 3단계 (12주차): 별도 서버(FastAPI)의 REST API
import requests
API = st.secrets.get("API_URL", "http://localhost:8000")
resp = requests.post(f"{API}/predict", json={
    "contract": contract, "tenure": tenure, "monthly": monthly,
    "internet": internet, "support": support}, timeout=5)
resp.raise_for_status()
r = resp.json()           # {"score": .., "level": .., "reasons": [...]}
```

용어

뜻

REST API	URL + HTTP 메서드(GET/POST)로 함수를 호출하는 약속
요청/응답	JSON(dict)을 보내고 JSON을 받는다
프론트엔드	오늘 만든 Streamlit 화면
백엔드	모델을 들고 있는 서버 (11~12주차 FastAPI)
secrets	URL·키 같은 값은 코드가 아니라 <code>.streamlit/secrets.toml</code> (Git 제외, 수업용 키만)

설계의 이득

응답 형식(score·level·reasons)을 지금 고정해 두면 3단계에서 화면을 한 줄도 안 고친다 . 기획서의 "2·3단계 연결 아이디어" 칸이 이것.

04

PART 4

실행과 시연 준비

"내 터미널에서는 되는데"를 없앤다 — 실행 위치·경로·패키지, 되는 버전의 스냅샷, 그리고 앱 전용 터미널

이 파트에서

- 1 재현 가능한 실행 — 상대 경로 · requirements.txt · 실행 위치
- 2 로컬 Git 여섯 명령 — 되는 버전의 스냅샷
- 3 LAB 3-6 앱 전용 터미널에서 실행 → 내 앱 주소
- 4 시연 체크리스트와 흔한 오류

재현 가능한 실행 — "내 터미널에서는 되는데요"를 없애는 법

전원이 같은 서버·같은 파이썬을 쓰므로 남는 원인은 세 가지: 실행 위치 · 경로 · 패키지

실행

```
# 1) 실행 위치 = app.py 가 있는 폴더 (~/.work 안)
cd ~/.work/churn_check
python check_env.py          # config.toml · 내 앱 주소 확인 (1주차 스크립트)

# 2) 쓰는 패키지를 파일로 적어 둔다 (재설치·3단계 이관용)
cat requirements.txt
# streamlit>=1.40
# pandas>=2.0
pip install -r requirements.txt  # 없는 것만 설치된다 (내 계정에 유지)

# 3) 실행
streamlit run app.py          # 브라우저: 내 앱 주소 .../proxy/8501/
```

실무에서 이렇게 틀린다 ⑤

AI에게 "requirements.txt 만들어 줘"라고 하면 **안 쓰는 패키지까지** 넣는 경우가 많다(예: matplotlib, scikit-learn). 안 쓰는 패키지가 적히면 재설치·3단계 이관 때 실패 원인이 된다 — import 한 것만.

깨지는 이유

예방

다른 폴더에서 실행

FileNotFoundError: data/... — 항상 app.py 폴더에서 streamlit run

절대 경로

/home/.../work/churn_check/data.csv 금지 → data/... 상대 경로

내 계정에 없는 패키지

ModuleNotFoundError → pip install <패키지> (내 계정에 유지) + requirements.txt에 기록

포트 충돌

Port 8501 is already in use → 내 이전 앱 Ctrl+C 또는 pkill -f "streamlit run"

비밀키 하드코딩

st.secrets + .streamlit/secrets.toml (Git에 넣지 않음)

로컬 Git 여섯 명령 — "되는 버전"의 스냅샷과 되돌리기

GitHub는 쓰지 않는다(계정 불필요). 내 폴더 안의 Git 이력만으로 2주차 v1~v2 저장을 정식으로 바꾼다

Git

```
git init                    # 폴더를 저장소로 (한 번)
git config user.name "홍길동" # 이 저장소에서만 (--global 은 ~/.gitconfig =
~/work 밖 → 로그아웃 시 사라짐)
git config user.email "22123456@suwon"
```

```
git add .                  # 바뀐 파일 담기 (.gitignore 제외)
git commit -m "3화면 프로토타입" # 스냅샷 (4주차부터 US-N: 접두어)
git status                 # 지금 무엇이 바뀌었나
git log --oneline          # 지금까지의 스냅샷
git diff                   # 마지막 커밋 이후 바뀐 줄
git restore app.py         # 파일 하나를 마지막 커밋으로
```

용어

뜻

저장소(repo)

프로젝트 폴더 + 전체 변경 이력 (.git/ 폴더 안)

커밋

"되는 버전"의 스냅샷. 4주차부터 메시지 앞 US-N:으로 명세와 연결

되돌리기

git restore <파일> — AI가 망친 파일을 마지막 커밋으로 (더 이전은 git log의 해시로)

.gitignore

이력에 넣지 않을 것: __pycache__/, .streamlit/secrets.toml, app.log, *.pyc

원격 없음

push·pull·브랜치·PR은 이 과목에서 쓰지 않는다. 4주차 "커밋 리듬·되돌리기·짹 리뷰"

루프와 Git

AI에게 파일을 고치라고 하기 전에 커밋. "잘 되던 게 갑자기 안 된다"의 해답이 git restore 한 줄이 된다. Claude Code는 git add-commit을 대신 해 주기도 하지만 되돌리는 결정은 사람이.

LAB 3-6 · 내 앱 주소로 열기 — 앱 전용 터미널

터미널 하나는 앱 전용으로 두고 다른 명령은 새 터미널에서. 내 앱 주소에서 3화면이 돌면 끝 (15분)

1

cd ~/work/churn_check
check_env.py 로 주소 확인

2

앱 전용 터미널
streamlit run app.py

3

내 앱 주소
.../proxy/8501/ 새 탭

4

3화면 클릭
시나리오 입력값으로

5

홈 화면 캡처·폴더
경로를 LMS에

명령

```
cd ~/work/churn_check
streamlit run app.py      # 이 터미널은 앱 전용 - 닫지 말고 둔다
# 브라우저: https://ahnbi4.suwon.ac.kr:5300/user/<아이디>/proxy/8501/
# (터미널의 localhost:8501 은 서버 안 주소 - 열지 않는다)

# 다른 명령은 새 터미널(런처 → Terminal)에서
# 앱 종료: 앱 터미널에서 Ctrl+C   · 남은 앱: pkill -f "streamlit run"
# pages/ 파일을 추가했으면 Ctrl+C 후 다시 streamlit run
# 3일 자동 로그아웃 뒤: 로그인 → 서버 시작 → 다시 실행 (파일은 그대로)
```

실행 후 확인

실패하면

3화면이 메뉴에 뜨나

pages/ 폴더가 있나 · 실행 위치가 app.py 폴더인가

데이터 화면 지표 7,043

data/telco_churn.csv 상대 경로 · 앱 터미널의
Traceback

위험 체크 제출 → 85점

core.py 같은 폴더 · ModuleNotFoundError → pip
install

주소가 404·빈 화면

앱 터미널이 살아 있나 · 주소 끝 / · 내 계정으로 로그인
한 브라우저인가

코드를 고쳤는데 그대로

브라우저 "Rerun" · pages/ 파일 추가는 재시작 필요

내 앱 주소는 내 계정으로 로그인한 브라우저에서만 열린다 — 짝·교수에게는 내 자리 화면으로 보여 준다(5주차 갤러리도 자기 자리). 오늘 홈 화면 캡처 + 폴더 경로를 LMS에. 5주차 시연은 이 폴더·이 명령 그대로(루브릭: 시연 시간에 앱이 안 돌고 백업 영상도 없으면 동작·완성도 최대 2점).

05

PART 5

내 주제 확정과 정리

수업 후 과제: 프로젝트 기획서 1장 — 좋은 주제의 조건 네 가지

이 파트에서

- 1 좋은 주제의 조건 — 시연 가능 · 데이터 · 예측
연결
- 2 프로젝트 기획서 1장 (템플릿 01)
- 3 한 장 요약 · 셀프 체크
- 4 과제와 4주차 예고

좋은 주제의 조건 — 5주 안에 시연되고, 2·3단계에 이어지는 것

아이디어의 크기가 아니라 "구체적인 한 사람의 문제"가 점수를 만든다

조건	질문	예 (통과 / 탈락)
① 5주차에 시연 가능	Must 3개를 4주차 안에 만들 수 있나? 외부 계정·승인이 필요한가?	통과: 공개 CSV 기반 추천 / 탈락: 실시간 결제·로그인 필수
② 데이터가 있다	공개 데이터·직접 입력·무료 API 중 무엇인가? 지금 당장 받을 수 있나?	통과: 공공데이터포털·Kaggle·UCI / 탈락: "학교에서 받을 예정"
③ 예측 기능이 붙는다	"입력 → 예측값" 하나를 말할 수 있나? (2단계 모델, 3단계 API)	통과: 매물 정보 → 월세 예측 / 탈락: 단순 게시판
④ 사용자가 구체적	"누가, 언제, 왜" 쓰는지 한 사람으로 말할 수 있나?	통과: 자취 2년차 식비 관리 / 탈락: "모든 사람"

데이터 출처 예 공공데이터포털(data.go.kr) · 서울열린데이터 · Kaggle · UCI · 교내 설문(직접 수집) · 수업 제공 데이터 6종 (data/)

주제 예 학식 혼잡 예측 · 중고 노트북 가격 추정 · 자취 식비 관리 · 강의평 요약 · 통신사 이탈 관리(오늘 앱 확장)

실무에서 이렇게 틀린다 ⑥

"AI로 뭐든 만들 수 있으니 크게" — 4주차 워크숍에서 가장 많이 줄여지는 기획서. 혼자 만드니 **Must 3개·화면 3개·데이터 확보 완료**가 통과선.

프로젝트 기획서 1장 — 수업 후 제출 (개인)

templates/01_프로젝트_기획서_1page.md. 4주차 첫 10분에 교수 피드백을 받는 문서

항목	쓸 것
이름 / 아이디	이름 · 학번 · 서버 아이디(262mb+ 학번) · 프로젝트 폴더(~/work/...)
서비스 한 줄	"__를 위해 __를 해 주는 서비스"
대상 사용자 · 문제	구체적인 한 사람 · 지금은 어떻게 하고 무엇이 불편한가
핵심 기능 3개 (Must)	5주차에 시연할 것만. 각 기능은 사용자 스토리 1문장 + 만드는 순서
화면 목록 · 데이터	3개 화면 이름 · 데이터 출처(링크)와 확보 여부
2·3단계 연결	"입력 → 예측값" 한 줄
리스크 1개와 대안	예: API 키가 안 나오면 → 샘플 CSV로 대체

지금 15분: 내 아이디어 2개를 조건 4개로 거른다(5분) → 짝과 교환해 서로 점검·질문(2분씩) → 1개 선택, 기획서 항목 채우기 시작. 못 정하면 **오늘 앱(이탈 위험 체크)의 확장**을 기본 주제로 가져간다.

한 장 요약 · 셀프 체크

왼쪽은 오늘의 표, 오른쪽은 옆 사람과 확인할 다섯 질문

주제	핵심
범위	Must 3개 · 화면 3~5개 · Won't 명시
3층 구조	입력(위젯) → 처리(core.py) → 출력. core는 streamlit을 모른다
화면 설계	텍스트 와이어프레임 → 계획 프롬프트 → 단계별 생성
상태	재실행 모델 · session_state · form · cache_data
다중 화면	pages/ 폴더, 숫자 접두어
실행·시연 준비	실행 위치·상대 경로·requirements(pip install) · 로컬 Git 6명령 · 앱 전용 터미널 → 내 앱 주소

- Q1. 버튼 안에서 그린 결과가 다음 클릭에 사라지는 이유와 해법은?
- Q2. core.py에 import streamlit을 넣지 않는 이유 둘.
- Q3. @st.cache_data를 붙이면 안 되는 함수는?
- Q4. 내 앱 주소가 404다. 확인할 것 셋은?
- Q5. 내 주제의 "입력 → 예측값"을 한 문장으로.

Q1 재실행으로 변수 초기화 → session_state에 저장하고 버튼 밖에서 그리기 / Q2 테스트 가능·API로 이전 가능 / Q3 업로드 파일·난수·현재 시각처럼 같은 인자에 다른 결과가 나오는 함수 / Q4 앱 터미널이 살아 있나(Traceback) · 주소 끝 /까지 정확한가 · 내 계정으로 로그인한 브라우저인가

과제 — 4주차 수업 전까지 (전부 개인)

4주차는 워크숍이라 수업 시간에 과제를 만들 시간이 없다

#	구분	과제	제출물 · 기준
1	개인	LAB 3-3~3-5 앱을 내 앱 주소에서 3화면 확인 + 커밋	홈 화면 캡처(주소창 포함) + 폴더 경로 (LMS) · git log 1개 이상
2	개인	프로젝트 기획서 1장 (템플릿 01)	md 또는 pdf — 조건 4개 통과, 데이터 링크 포함
3	개인	내 폴더 정리: git init + 저장소 설정(user.name) + README-requirements.txt·gitignore·AI_LOG.md(템플릿 03) 커밋	git log 커밋 2개 이상
4	개인	데이터 확보: 실제 파일을 data/에 넣고 df.shape 열 목록을 README에	데이터 없으면 4주차 시작 전 주제 변경

다음 주

4주차 예고: **개인 프로젝트 개발 워크숍** — 기능 명세서(0:40 제출) → 개발 → 짝 데모(1:20) → 교수 피드백. 이론은 30분(사용자 스토리·MoSCoW, 커밋 리듬·되돌리기·짝 리뷰, AI 규칙 파일 [CLAUDE.md](#), 품질 체크리스트). 데이터·기획서를 전원 준비.

- 참고: Streamlit Docs — "Multipage apps", "Session State", "Caching", "Configuration" · Pro Git 2판 1~2장(무료) · Claude Code Docs "Common workflows"